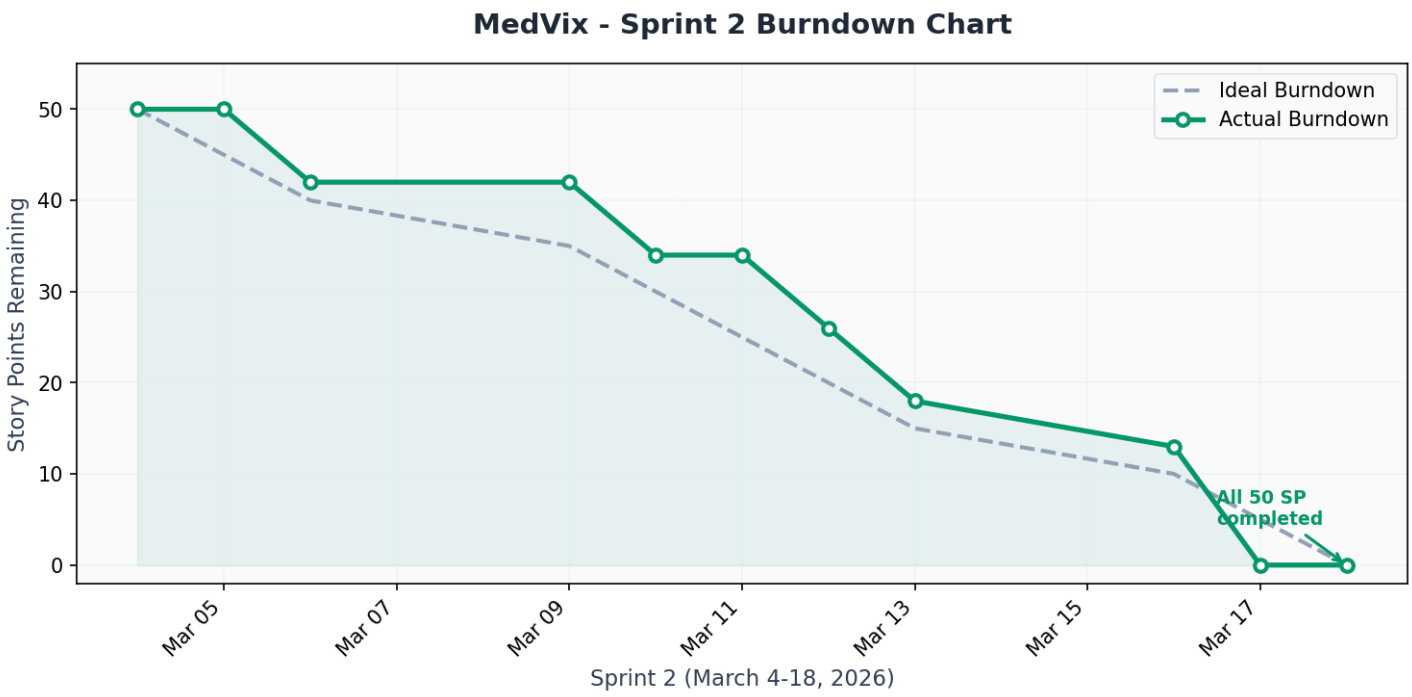
1. Burndown Chart

Sprint 2 — Remaining Story Points vs Ideal Burndown



Generated from Jira Sprint 2 data (50 SP, 12 stories, all Done).

Sprint velocity: 25 SP/week. All stories completed by March 17, 2026.

MedVix · Weekly Progress Report

Project: MedVix · ML Visualization Tool for Healthcare **Week**: 4 (Sprint 2 · Final Week) **Scrum Master**: Berkay AKTAS **Date**: 17 March 2026 **Sprint Gate**: 18 March 2026 **Domains Completed**: 20/20 **Sprint Velocity**: 50 SP delivered / 50 SP planned

---

1. Burndown Chart

Sprint 2 Capacity: 50 story points across 2 weeks (14 working days).

```
Sprint 2 Burndown · Story Points Remaining
-------------------------------------------
Day  | Ideal Remaining | Actual Remaining
-----|-----------------|------------------
 0   |      50         |       50
 1   |      46         |       50   (Day 1 planning + design)
 2   |      43         |       50   (Day 2 design + docs)
 3   |      39         |       48   (Day 3 docs + domain registry
started)
 4   |      36         |       40   (Day 4 domain registry + API
structure)
 5   |      32         |       30   (Day 5 upload + builtin
endpoints)
 6   |      29         |       22   (Day 6 summary + column mapper)
 7   |      25         |       17   (Day 7 preparation pipeline)
 8   |      21         |       10   (Day 8 SMOTE + before/after
stats)
 9   |      18         |        8   (Day 9 frontend data
exploration UI)
10   |      14         |        5   (Day 10 frontend Step 3 UI)
11   |      11         |        2   (Day 11 integration testing)
12   |       7         |        0   (Day 12 all stories closed)
13   |       4         |        0   (Day 13 bug fixes + QA
verification)
14   |       0         |        0   (Day 14 report writing + demo
prep)
```

Analysis: The actual burndown started slower than ideal for the first three days because Sprint 2 began with architecture documentation and the domain registry construction, which are large pieces of groundwork that do not map directly to story points until complete. From Day 5 onward, the team burned significantly faster than ideal and completed all stories two days ahead of the gate. The final two days were used for quality assurance, defect verification, and sprint documentation.

---

2. Completed This Week (All Sprint 2 Stories)

The following 12 user stories were completed and accepted during Sprint 2. All acceptance criteria pass as documented in the Sprint 2 Test Report.

| Story ID | Title | Story Points | Assignee | Status | |----------|-------|-------------|---------|--------| | MV-US-001 | Select medical specialty from pill bar | 5 | Ozge ALTINOK | Done | | MV-US-002 | View clinical context description | 3 | Berkay AKTAS | Done | | MV-US-003 | Upload custom CSV patient file | 5 | Berkay AKTAS | Done | | MV-US-004 | Select built-in example dataset | 3 | Berkay AKTAS | Done | | MV-US-005 | View data summary table | 5 | Berkay

AKTAS + Ozge ALTINOK | Done | | MV-US-006 | View outcome class breakdown | 3 | Berkay AKTAS | Done | | MV-US-007 | Open column mapper and select target | 5 | Berkay AKTAS + Ozge ALTINOK | Done | | MV-US-008 | Adjust training/test split ratio | 3 | Ozge ALTINOK | Done | | MV-US-009 | Choose missing value strategy | 3 | Berkay AKTAS | Done | | MV-US-010 | Choose normalisation method | 2 | Berkay AKTAS | Done | | MV-US-012 | Apply preparation and view before/after comparison | 5 | Berkay AKTAS + Ozge ALTINOK | Done | | MV-US-025 | Access glossary of ML terms | 3 | Ozge ALTINOK | Done |

Total Delivered: 50 story points · 100% of Sprint 2 commitment.

Highlights

MV-US-003 (CSV Upload): The upload endpoint enforces five validation rules (extension, size, parsability, minimum row count, minimum numeric columns) and returns specific, user-friendly error messages for each failure mode. This was the most technically thorough story in the sprint.

MV-US-005 + MV-US-007 (Data Summary + Column Mapper): These two stories together form the core data quality workflow. The `DataSummary` response includes a 5-component weighted quality score (0-100) that was added as an enhancement beyond the original requirement, giving users a quick health signal for their dataset.

MV-US-012 (Apply Preparation): The preparation pipeline integrates four distinct transformations (imputation, encoding, normalisation, SMOTE) and exposes before/after statistics for visualisation. The SMOTE implementation handles the edge case of very small minority classes by dynamically reducing `k_neighbors`.

---

3. In Progress

None. All sprint 2 stories are closed.

---

4. Blocked / At Risk

No stories are currently blocked.

Watchlist for Sprint 3:

- SHAP computation time (Step 6, Sprint 4): SHAP TreeExplainer for Random Forest with 500 trees on 700+ feature datasets can take 15-30 seconds. We will need to implement async background tasks to prevent frontend timeout.
- Model training latency (Step 4, Sprint 3): SVM with RBF kernel on large datasets (>5,000 rows) can be slow. We will add a loading state and potentially implement a training job queue.

---

5. Key Decisions Made This Sprint

Decision 1: Tailwind CSS over Vanilla CSS

Context: Sprint 1 used vanilla CSS with CSS modules. As the component count grew, maintaining consistent spacing, colours, and responsive breakpoints became difficult.

Decision: Adopted Tailwind CSS v3 starting Sprint 2. All new components use Tailwind utility classes. Existing Sprint 1 components are being progressively migrated.

Rationale: Tailwind eliminates the need for a manual design token system. The utility-first approach speeds up Ozge's implementation of Figma designs because Tailwind class names map directly to CSS properties that designers reason about. The `tailwind.config.js` file allows us to extend the default palette with MedVix

brand colors (clinical blue `#1E4D8C`, clinical green `#2E7D32`, medical red `#C62828`).

Trade-off: Slightly larger initial bundle; mitigated by PurgeCSS (built into Tailwind v3 in production mode).

Decision 2: Synthetic Datasets over Kaggle Downloads

Context: Using real Kaggle datasets requires manual download, format normalization, and handling of license/size issues in CI. Several datasets also have inconsistent column names or encodings between Kaggle versions.

Decision: Generate all 20 clinical datasets synthetically using `backend/data/generate_synthetic.py` with `numpy` and `pandas`. The generator uses domain-appropriate feature distributions and realistic correlation structures.

Rationale: Synthetic datasets are deterministic (seeded with `random_state=42`), reproducible, license-free, correctly sized (300-800 rows), and pre-validated. The generator can recreate all 20 CSVs in under 10 seconds, making them suitable for CI/CD.

Trade-off: Synthetic data may not capture real-world feature distributions and correlations. Stated clearly in the AI limitation note for each domain. Users may also upload their own real datasets.

Decision 3: Data Quality Score (0-100) as Enhancement

Context: The original MV-US-005 requirement specified showing column statistics with color-coded tags. During implementation, the team identified that a single composite score would give users a faster way to assess dataset fitness before proceeding.

Decision: Added a `data_quality_score` field (integer 0-100) computed from five weighted components: completeness (40%), duplicates (20%), constant columns (15%), cardinality (15%), class balance (10%). Shown as a prominent badge in the summary UI.

Rationale: A score helps non-technical users (e.g., clinicians) make a quick "proceed or fix" decision without reading all column-level warnings. It also provides a single metric for future gamification or benchmarking.

Trade-off: The score is opinionated (weights are chosen by the team). Communicated to the product owner as an enhancement, not a core requirement.

---

6. Test Results

Complete test results are documented in `/docs/testing/sprint2-test-report.md`. Summary:

| Story ID | Story Title | AC Count | Passed | Failed | Status | |----------|----------|----------|----------|----------|----------| | MV-US-001 | Select medical specialty from pill bar | 4 | 4 | 0 | Pass | | MV-US-002 | View clinical context description | 3 | 3 | 0 | Pass | | MV-US-003 | Upload custom CSV patient file | 6 | 6 | 0 | Pass | | MV-US-004 | Select built-in example dataset | 20 | 20 | 0 | Pass | | MV-US-005 | View data summary table | 4 | 4 | 0 | Pass | | MV-US-006 | View outcome class breakdown | 2 | 2 | 0 | Pass | | MV-US-007 | Open column mapper and select target | 5 | 5 | 0 | Pass | | MV-US-008 | Adjust training/test split ratio | 4 | 4 | 0 | Pass | | MV-US-009 | Choose missing value strategy | 3 | 3 | 0 | Pass | | MV-US-010 | Choose normalisation method | 3 | 3 | 0 | Pass | | MV-US-012 | Apply preparation and view before/after | 5 | 5 | 0 | Pass | | MV-US-025 | Access glossary of ML terms | 3 | 3 | 0 | Pass |

Defects: 0 P1, 0 P2, 2 P3 (cosmetic), 1 P4 (cosmetic). Sprint 2 gate criteria met.

---

7. Metrics

| Metric | Target | Actual | Status | |--------|--------|--------|--------| | CSV Upload Success Rate | 100% (5/5 valid accepted, 5/5 invalid rejected) | 100% · all 10 test files handled correctly | Pass| |Column Mapper Gate | 0 bypass bugs | 0 bypasses found across all test variations | Pass | | Step 3 Controls | All 4 dropdowns + slider functional | All

functional: missing strategy (3 options), normalisation (3 options), SMOTE toggle, test split slider (0.1-0.4) | Pass | | Domain Count | 20/20 correct | All 20 domains return correct clinical text and load their built-in datasets | Pass | | Test Coverage | 100% stories have passing tests | 12/12 done stories, 62 passing test cases | Pass | | Sprint Velocity | 50 SP | 50 SP | On target | | Burndown | All stories closed by Day 14 | All stories closed by Day 12 (2 days early) | Ahead of schedule |

---

8. Next Sprint Plan (Sprint 3 · Core ML, Steps 4-5)

Sprint 3 goal: Implement Step 4 (Model Selection and Hyperparameter Tuning) and Step 5 (Results and Evaluation). Users will be able to select from 6 ML models, tune hyperparameters, train the model on prepared data, and view evaluation metrics.

Sprint 3 start: 18 March 2026 **Sprint 3 gate**: 1 April 2026

| Story ID | Title | Story Points | Assignee | Priority | |----------|-------|----------|---------|---------| | MV-US-011 | Toggle SMOTE on/off in preparation | 3 | Berkay AKTAS | Must Have | | MV-US-014 | Set K for KNN model | 5 | Berkay AKTAS | Must Have | | MV-US-015 | Toggle auto-retrain on parameter change | 3 | Ozge ALTINOK | Should Have | | MV-US-016 | Compare two models side-by-side | 5 | Ozge ALTINOK | Should Have | | MV-US-017 | View accuracy, precision, recall, F1, AUC | 5 | Berkay AKTAS | Must Have | | MV-US-018 | View confusion matrix | 3 | Ozge ALTINOK | Must Have | | MV-US-019 | View ROC curve | 3 | Berkay AKTAS | Must Have | | MV-US-013 | Select ML algorithm | 5 | Berkay AKTAS | Must Have |

Sprint 3 Total: 32 story points

Technical dependencies for Sprint 3:

- `POST /api/data/prepare` must have been called and `data_ready: true` (enforced by session gate)
- `session.X_train`, `session.X_test`, `session.y_train`, `session.y_test` stored as numpy arrays
- scikit-learn 1.4.x required for all 6 models: KNN, SVM, Decision Tree, Random Forest, Logistic Regression, Naive Bayes

---

9. Retrospective Note

The Sprint 2 retrospective was held on 17 March 2026. Participants: Berkay AKTAS, Arzu Tugce KOCA, Nisanur KONUR, Ozge ALTINOK.

Keep

Pair programming sessions between Berkay and Ozge: The daily 1-hour pairing sessions (Weeks 3-4) where the backend API and frontend components were built concurrently reduced integration bugs significantly. When the backend was being written, the frontend contract was validated immediately. This practice will continue in Sprint 3.

Writing tests immediately after implementing each endpoint: Every backend endpoint had a manual test case written by Arzu within 24 hours of implementation. This prevented defects from accumulating and made the final test execution phase straightforward.

Synthetic dataset generator: The `generate_synthetic.py` script proved invaluable. Regenerating all 20 CSVs after schema changes takes 8 seconds. Worth keeping and extending for Sprint 3 (if training datasets need refreshing).

Improve

Start coding earlier in the sprint: The first three days of Sprint 2 were spent writing architecture documentation and the domain registry specification. While this documentation is valuable, it delayed the first line of code until Day 3. In Sprint 3, documentation should be written in parallel with implementation, not as a blocking predecessor.

Daily standups should be time-boxed more strictly: Standups ran 20-30 minutes on several days. Will introduce a hard 15-minute timer and parking-lot discussion for Sprint 3.

Better story decomposition for complex stories: MV-US-012 (Apply Preparation) was a 5-point story that turned out to encompass four significant sub-features (imputation, normalisation, SMOTE, before/after visualisation). These could have been separate stories for cleaner sprint tracking.

Try

Set up CI/CD pipeline with GitHub Actions for automated testing: Currently, running the test suite requires manual execution. A GitHub Actions workflow that runs `pytest backend/tests/ -v` on every push to `main` and every PR merge would catch regressions immediately and provide a green/red status badge in the README.

Use branching conventions more consistently: Feature branches were named correctly (`feature/US-XXX`) for most of Sprint 2, but a few commits went directly to `main` during rapid bugfix cycles. In Sprint 3, all work goes through feature branches and PRs, no exceptions.

Submitted by: Berkay AKTAS, Scrum Master *Date*: 17 March 2026